

Customer Shopping Behavior Analysis

End-to-End Data Analytics Project Report

3,900
Purchases

18
Columns

37
Missing ratings

1. Project Overview

This project analyzes customer shopping behavior using transactional data across multiple product categories. The objective is to identify spending patterns, customer segments, product preferences, discount behavior and subscription trends, and to turn those findings into actionable business recommendations.

2. Dataset Summary

Dimension	Details
Rows	3,900 transactions
Columns	18
Customer data	Age, Gender, Location, Subscription Status
Purchase data	Item Purchased, Category, Purchase Amount, Season, Size, Color
Behavior data	Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type
Data quality	37 missing values in Review Rating

3. Analytical Workflow

- Python - data loading, exploration, cleaning, feature engineering and database loading.
- SQL Server - business-oriented aggregation and transaction analysis.
- Power BI - interactive visualization and dashboard presentation.

The report presents the project as a practical end-to-end analytics workflow, supported by evidence from the Python notebook, SQL Server analysis, and Power BI dashboard.

02 Data Preparation & Exploratory Analysis

Python stage - cleaning and preparing the dataset for analysis

The project workflow begins in Python, where the raw transaction data is prepared before database analysis. The process focuses on structure, data quality, consistency and creating useful analytical features.

Step	Work completed
Data loading	Imported the dataset using pandas.
Initial exploration	Reviewed structure with <code>df.info()</code> and descriptive statistics with <code>.describe()</code> .
Missing values	Checked nulls and imputed missing Review Rating values using the median rating of the relevant product category.
Standardization	Renamed columns to snake_case for readability and documentation.
Feature engineering	Created <code>age_group</code> and <code>purchase_frequency_days</code> fields.
Consistency check	Compared <code>discount_applied</code> and <code>promo_code_used</code> for redundancy and removed <code>promo_code_used</code> .
Database integration	Loaded the cleaned DataFrame into the SQL database for business analysis.

Analytical logic

- Age grouping supports demographic comparison at a level suitable for dashboard reporting.
- Purchase-frequency information supports customer segmentation and repeat-buyer analysis.
- Cleaning before SQL reduces inconsistencies and makes the downstream business questions easier to reproduce.

The supplied project brief records 37 missing Review Rating values and specifies category-level median imputation as the treatment.

Python Evidence - Notebook Walkthrough

The screenshots below document the Python stage performed in my own Jupyter notebook. They show the dataset structure, data-quality checks, feature engineering, consistency checks and the database connection used before SQL analysis.

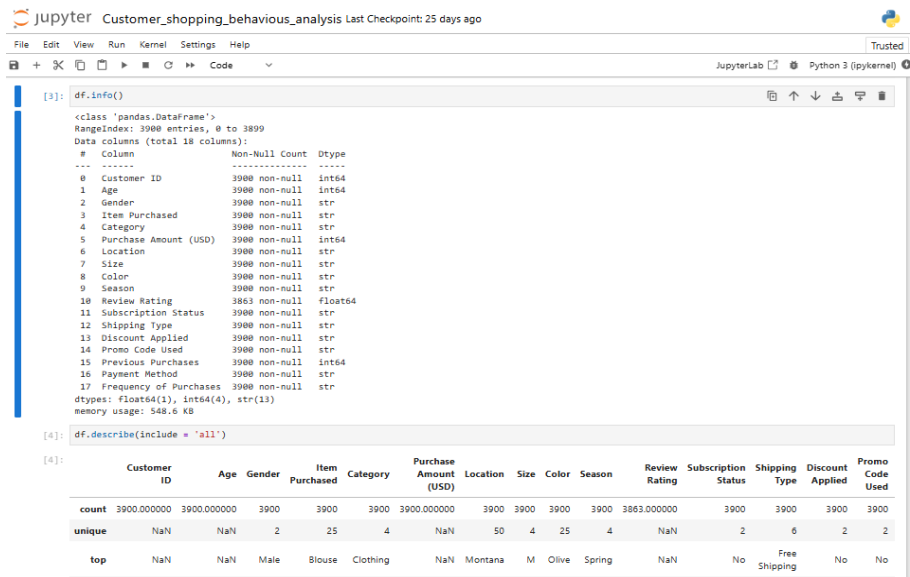


Figure 3.1 - Dataset structure and descriptive profiling in Jupyter.

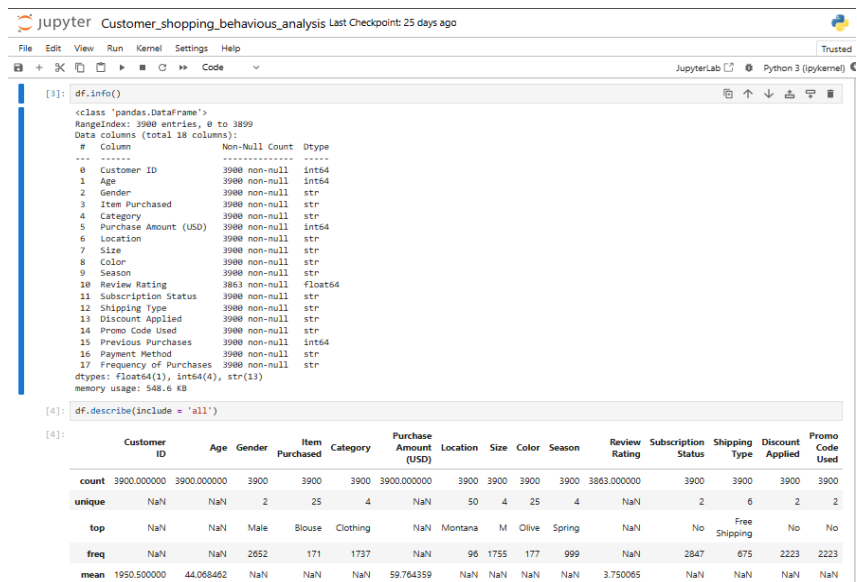
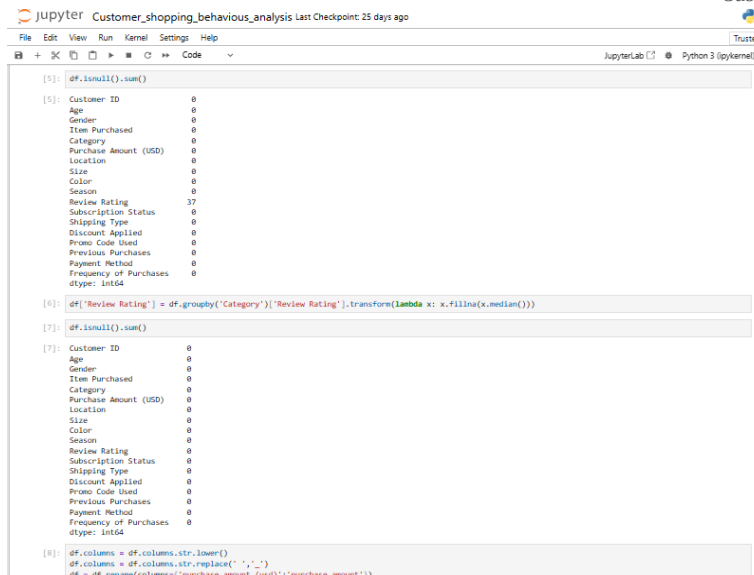


Figure 3.2 - Initial dataset inspection and descriptive statistics.

Initial inspection of the 3,900-row dataset and its 18 columns.



```

[5]: df.isnull().sum()

Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD)  0
Location         0
Size            0
Color           0
Season          0
Review Rating    37
Subscription Status  0
Shipping Type    0
Discount Applied  0
Promo Code Used  0
Previous Purchases  0
Payment Method   0
Frequency of Purchases  0
dtype: int64

[6]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))

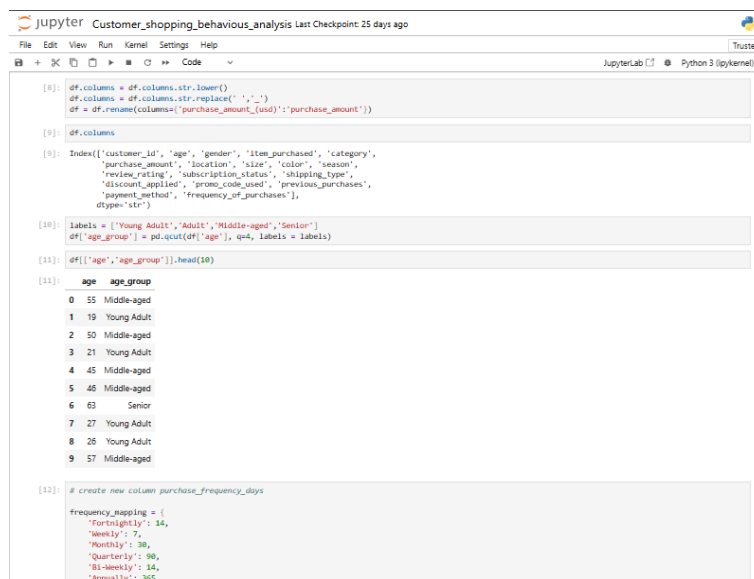
[7]: df.isnull().sum()

Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD)  0
Location         0
Size            0
Color           0
Season          0
Review Rating    0
Subscription Status  0
Shipping Type    0
Discount Applied  0
Promo Code Used  0
Previous Purchases  0
Payment Method   0
Frequency of Purchases  0
dtype: int64

[8]: df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_usd':'purchase_amount'})

```

Figure 3.3 - Missing-value check and review-rating imputation.



```

[8]: df.columns = df.columns.str.lower()
df.columns = df.columns.str.replace(' ', '_')
df = df.rename(columns={'purchase_amount_usd':'purchase_amount'})

[9]: df.columns

Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
      'purchase_amount', 'location', 'size', 'color', 'season',
      'review_rating', 'subscription_status', 'shipping_type',
      'discount_applied', 'promo_code_used', 'previous_purchases',
      'payment_method', 'frequency_of_purchases'],
      dtype='object')

[10]: labels = ["Young Adult", "Adult", "Middle-aged", "Senior"]
df['age_group'] = pd.cut(df['age'], q=4, labels = labels)

[11]: df[['age', 'age_group']].head(10)

   age  age_group
0   55  Middle-aged
1   19  Young Adult
2   50  Middle-aged
3   21  Young Adult
4   45  Middle-aged
5   46  Middle-aged
6   63    Senior
7   27  Young Adult
8   26  Young Adult
9   57  Middle-aged

[12]: # create new column purchase_frequency_days

frequency_mapping = {
    "fortnightly": 14,
    "weekly": 7,
    "monthly": 30,
    "quarterly": 90,
    "bi-weekly": 14,
    "annually": 365,
}

```

Figure 3.4 - Age-group feature engineering.

Customer ages are transformed into Young Adult, Adult, Middle-aged and Senior groups.

The JupyterLab interface displays the following code and data:

```
[13]: df[['purchase_frequency_days', 'frequency_of_purchases']].head(10)
```

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

```
[14]: df[['discount_applied', 'promo_code_used']].head(10)
```

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[15]: (df[['discount_applied'] == df['promo_code_used']]).all()
[15]: np.True_
[16]: # Dropping promo code used column
```

Figure 3.5 - Purchase-frequency mapping and redundancy check.

The JupyterLab interface displays the following code:

```
[17]: (df[['discount_applied'] == df['promo_code_used']]).all()
[15]: np.True_
[16]: # Dropping promo code used column
df = df.drop('promo_code_used', axis=1)
[17]: df.columns
[17]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
'purchase_amount', 'location', 'size', 'color', 'season',
'review_rating', 'subscription_status', 'shipping_type',
'discount_applied', 'previous_purchases', 'payment_method',
'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
dtype='str')
[18]: pip install pyodbc sqlalchemy
Requirement already satisfied: pyodbc in C:\Users\shado\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages (5.3.0)
Requirement already satisfied: sqlalchemy in C:\Users\shado\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages (2.0.52)
Requirement already satisfied: greenlet>=1 in C:\Users\shado\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages (from sqlalchemy) (3.5.5)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\shado\AppData\Local\Python\pythoncore-3.14-64\lib\site-packages (from sqlalchemy) (4.16.0)
Note: you may need to restart the kernel to use updated packages.
[19]: import urllib
from sqlalchemy import create_engine

# 1. Exact SMS Server and Database name
server_name = r'localhost\SQLEXPRESS'
database_name = 'customer_behaviour'

# 2. Build the connection string for Windows Authentication
params = urllib.parse.quote_plus(
    f'DRIVER={{ODBC Driver 17 for SQL Server}};'
    f'SERVER={server_name};'
    f'DATABASE={database_name};'
    f'Trusted_Connection=yes;'
)

# 3. Fire up the engine!
engine = create_engine(f'mssqlpyodbc:///{dbms_connect={params}}')
print(f'Engine successfully created and connected to SMS!')
Engine successfully created and connected to SMS!
[20]: # Pushes DataFrame directly to a new SQL table named 'shopping_data'
df.to_sql('shopping_data', con=engine, if_exists='replace', index=False)
```

Figure 3.6 - SQL Server connection from Python.

The cleaned DataFrame is connected to SQL Server and prepared for database analysis.

03 SQL Analysis - Core Business Questions

SQL Server results captured from my current project

The database stage converts the cleaned data into business-facing measures. My SQL Server output shows the key result sets used for the report and dashboard.

The screenshot displays four SQL query result sets in a grid. The first table shows revenue by gender. The second table shows the top 5 products by rating. The third table compares shipping types. The fourth table shows subscription status and performance metrics.

	gender	revenue
1	Male	157890
2	Female	75191

	customer_id	purchase_amount
1	2	64
2	3	73
3	4	90
4	7	85
5	9	97
6	12	68
7	13	72
8	16	81

	item_purchased	Average Product Rating
1	Gloves	3.86
2	Sandals	3.84
3	Boots	3.82
4	Hat	3.8
5	Skirt	3.78

	shipping_type	Avg Purchase Amount
1	Express	60.480000
2	Standard	58.460000

	subscription_status	total_customers	avg_spend	total_revenue
1	No	2847	59.870000	170436.00
2	Yes	1053	59.490000	62645.00

Query executed successfully. | localhost\SQLEXPRESS (16.0 ... | DESKTOP-0N2Q9L7\shado ... | customer_behaviour | 00:00:01 | Row: 1, Col: 2 | 863 rows

Figure 1. SQL Server result sets: revenue by gender, top-rated products, shipping comparison and subscription performance.

Key findings from the captured output

Question	Observed result
Revenue by gender	Male customers generated 157,890 in revenue vs. 75,191 for female customers.
Top 5 products by rating	Gloves 3.86, Sandals 3.84, Boots 3.82, Hat 3.80, Skirt 3.78.
Shipping comparison	Express average purchase: 60.48; Standard: 58.46.
Subscribers vs non-subscribers	Non-subscribers: 2,847 customers, 59.87 average spend, 170,436 revenue. Subscribers: 1,053 customers, 59.49 average spend, 62,645 revenue.

04 SQL Analysis - Discounts & Customer Segmentation

Using transactional behavior to understand customer value

Results Messages

subscription_status	total_customers	avg_spend	total_revenue
No	2847	59.870000	170436.00
Yes	1053	59.490000	62645.00

item_purchased	discount_rate
Hat	50.00
Sneakers	49.66
Coat	49.07
Sweater	48.17
Pants	47.37

customer_segment	Number of Customers
New	83
Returning	701
Loyal	3116

item_rank	category	item_purchased	total_orders
1	Accessories	Jewelry	171
2	Accessories	Belt	161
3	Accessories	Sunglasses	161
4	Clothing	Blouse	171
5	Clothing	Pants	171
6	Clothing	Shirt	169
7	Footwear	Sandals	160
8	Footwear	Shoes	150

subscription_status	repeat_buyers
Yes	958
No	2518

Query executed successfully.localhost\SQLEXPRESS (16.0 ... DESKTOP-0N2Q9L7\shado ... customer_behaviour00:00:01Row: 1, Col: 2863 rows

Figure 2. SQL Server result sets: subscription comparison, discount-dependent products, customer segments, ranked products and repeat buyers.

Additional findings

Analysis	Observed result
Discount-dependent products	Hat 50.00%, Sneakers 49.66%, Coat 49.07%, Sweater 48.17%, Pants 47.37% discounted purchases.
Customer segmentation	New: 83; Returning: 701; Loyal: 3,116.
Repeat buyers	958 subscribers appear in the >5 purchase repeat-buyer group, compared with 2,518 non-subscribers in the same output.
Top products by category	Highest ranked examples include Jewelry, Belt, Sunglasses, Blouse, Pants, Shirt, Sandals, Shoes, Sneakers, Jacket and Coat.

Interpretation

The segment distribution is heavily concentrated in the Loyal group, while discount dependence is particularly high for several products. These findings are useful for designing retention offers without relying on blanket discounting.

05 SQL Analysis - Product & Age Insights

Comparing product demand and revenue contribution

customer_segment	Number of Customers
1 New	83
2 Returning	701
3 Loyal	3116

item_rank	category	item_purchased	total_orders
1	Accessories	Jewelry	171
2	Accessories	Belt	161
3	Accessories	Sunglasses	161
4	Clothing	Blouse	171
5	Clothing	Pants	171
6	Clothing	Shirt	169
7	Footwear	Sandals	160
8	Footwear	Shoes	150

subscription_status	repeat_buyers
1 Yes	958
2 No	2518

age_group	total_revenue
1 Young Adult	62143
2 Middle-aged	59197
3 Adult	55978
4 Senior	55763

Query executed successfully. | localhost\SQLEXPRESS (16.0 ... | DESKTOP-0N2Q9L7\shado ... | customer_behaviour | 00:00:01 | Row: 1, Col: 2 | 863 rows

Figure 3. SQL Server result sets including ranked products and revenue contribution by age group.

Revenue by age group

Age group	Total revenue
Young Adult	62,143
Middle-aged	59,197
Adult	55,978
Senior	55,763

What this adds to the analysis

- Young Adults are the highest-revenue age group in the captured result set.
- Middle-aged customers also make a meaningful contribution and sit close behind Young Adults.
- The age comparison can support targeted campaign planning rather than treating the customer base as homogeneous.

The product-ranking output also provides a compact way to identify the strongest-selling items within each category and prioritize those products in campaigns.

06 Power BI Dashboard

Interactive presentation layer built from the analyzed data

The final Power BI dashboard brings the analysis together through KPI cards, filters and category/age comparisons. The dashboard shown below is the version captured from my own work.

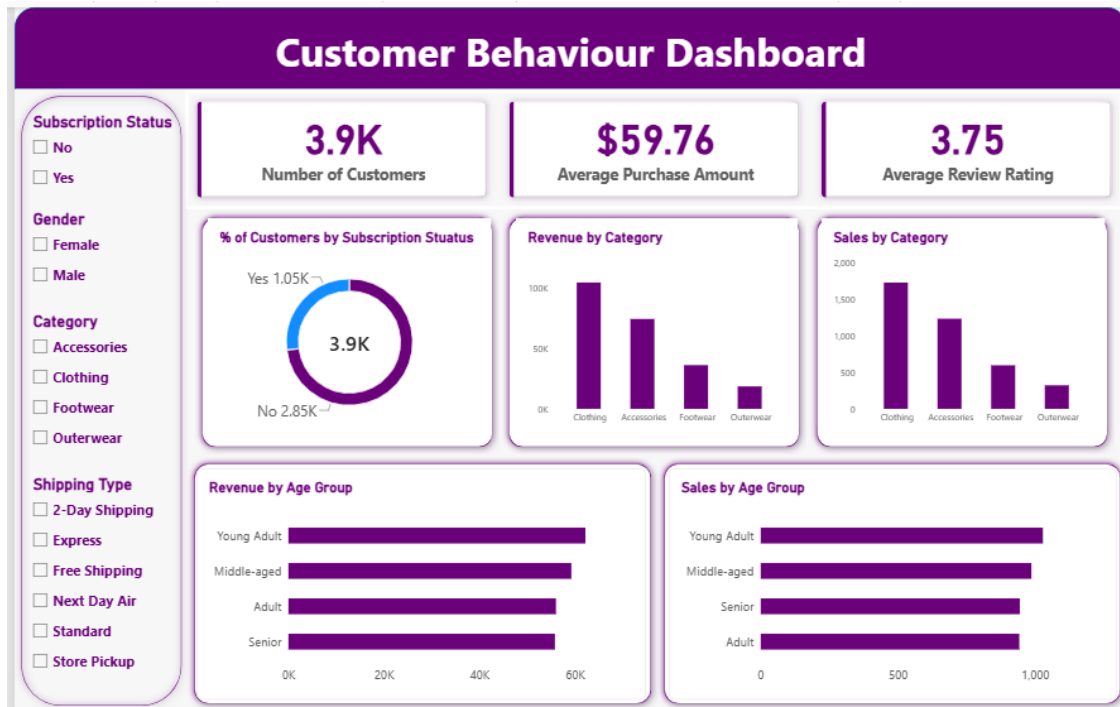


Figure 4. My Power BI Customer Behaviour Dashboard.

Dashboard elements

Component	Purpose
KPI cards	Number of customers, average purchase amount and average review rating.
Subscription filter & donut	Shows the customer mix between subscribed and non-subscribed customers.
Category visuals	Compares revenue and sales across Accessories, Clothing, Footwear and Outerwear.
Age-group visuals	Compares revenue and sales across Young Adult, Middle-aged, Adult and Senior customers.
Interactive slicers	Subscription Status, Gender, Category and Shipping Type allow focused exploration.

07 Business Recommendations

Turning the analysis into practical actions

Recommendation	Why it matters
Boost subscriptions	Promote exclusive benefits and clear value propositions for subscribers.
Strengthen loyalty programs	Reward repeat buyers and create pathways to move Returning customers into the Loyal segment.
Review discount policy	Use product-level discount dependence to balance sales stimulation against margin protection.
Position best products	Feature top-rated and best-selling products more prominently in campaigns and merchandising.
Target by age and fulfillment	Use higher-revenue age groups and shipping-behavior patterns to make marketing more focused.

Project conclusion

This project demonstrates an end-to-end analytics workflow: raw transactional data is prepared in Python, business questions are answered with SQL Server, and the results are communicated through Power BI. The strongest value of the project is not any single chart, but the connection between data preparation, reproducible SQL analysis and decision-oriented visualization.

Note on evidence

The screenshots in this report are taken from the actual project work completed in Jupyter, SQL Server, and Power BI.